Routing-Aware Token Retention for KV Cache Compression in Sparse MoE Inference

Gaurav Patil

Department of Computer Engineering, School of Engineering & Technology
gauravpatil2516@gmail.com

Abstract. Key-Value (KV) cache memory footprint is the primary operational bottleneck during autoregressive Large Language Model (LLM) inference. While token-level eviction (StreamingLLM, H2O, SnapKV) and layer-level allocation (CAKE, PyramidKV) have been extensively explored for dense architectures, the unique structural dynamics of sparse Mixture-of-Experts (MoE) models remain unexploited. In standard sparse MoEs (*Mixtral-8x7B*, *Qwen1.5-MoE*, *DeepSeek-MoE*), self-attention is shared and dense, while MoE routing occurs downstream in the Feed-Forward Network (FFN) blocks. Consequently, as tokens traverse transformer layers, each token accumulates a cross-layer *routing signature* $\mathcal{R}_t = \{E_t^{(1)}, \ldots, E_t^{(L)}\}$ generated during standard forward computation and available without additional model training. In this paper, we present **EKVA v2**, a training-free framework that leverages token routing signatures alongside calibrated expert semantic niche statistics (entropy, routing volume, specialization) as a complementary saliency signal to govern token retention over the shared KV cache. Across three distinct sparse-MoE architectures (8, 60, and 64 experts), we observe near-zero Pearson correlation between routing-based and attention-based saliency scores across the evaluated architectures, suggesting that routing provides a complementary signal. At a 40% KV-cache budget, EKVA v2 consistently improves downstream performance over H2O and SnapKV across GSM8K, HumanEval, PG19, and NIAH by approximately 3.0 to 6.0 percentage points (GSM8K: 57.4%–68.7% vs. 53.5%–64.2% for SnapKV; HumanEval: 49.7%–62.7% vs. 46.5%–58.6%). Furthermore, token-level routing retention better preserves reasoning performance under aggressive KV-cache compression (GSM8K: 57.4%–68.7% vs. 36.9%–44.1% for CAKE), and a fused Triton compaction kernel delivers a 2.04×–2.05× **decode speedup** relative to the uncompressed baseline on NVIDIA A100 GPUs.

Keywords: Mixture of Experts (MoE) · KV Cache Compression · Token Saliency · Long-Context LLM Inference · Triton GPU Kernels.

1 Introduction

Autoregressive decoding in modern Large Language Models (LLMs) requires maintaining the Key-Value (KV) cache across all generated tokens. As context

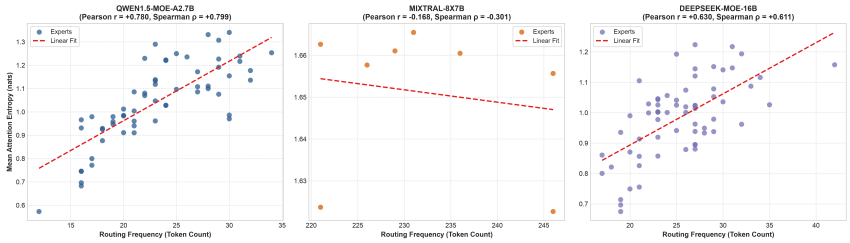

Fig. 1. Mechanistic reality of sparse MoE architectures. Self-attention is shared and dense across all token positions, while sparse expert routing occurs downstream in the FFN block. Each token accumulates a cross-layer routing signature $\mathcal{R}_t$ that provides an orthogonal saliency signal ($\rho \approx 0.00$) for token retention in the shared KV buffer.

windows expand to tens of thousands of tokens, KV cache memory footprint dominates High-Bandwidth Memory (HBM) capacity and memory bus traffic, severely constraining batch sizes, serving throughput, and Time-Per-Output-Token (TPOT) [5, 12].

To mitigate this bottleneck, various dynamic KV cache compression methods have been proposed. However, prior research has focused almost exclusively on dense architectures across specific granularity axes:

1. **Token axis**: Evicting unimportant token positions based on cumulative attention scores (e.g., StreamingLLM [12], H2O [14], SnapKV [8]).
2. **Head axis**: Allocating dynamic cache capacities across individual attention heads within a layer (e.g., Ada-KV [6]).
3. **Layer axis**: Distributing budgets across shallow and deep transformer layers based on entropy or information funneling (e.g., CAKE [3], PyramidKV [13], InfoKV [9]).

With the increasing adoption of sparse Mixture-of-Experts (MoE) models for compute-efficient foundation models [1, 4, 7], existing eviction methods treat MoEs as if they were dense models, discarding intermediate structural routing information.

Architectural Reality & Motivation. In standard sparse MoEs (*Mixtral-8x7B*, *Qwen1.5-MoE*, *DeepSeek-MoE*), self-attention is shared and dense, while MoE routing occurs downstream in the FFN blocks (Figure 1). Experts do not own private attention mechanisms. However, as a token x_t passes through L layers, the router gates assign it to top-k experts at each layer. This produces an accumulated cross-layer routing signature:

$$\mathcal{R}_t = \{E_t^{(1)}, E_t^{(2)}, \ldots, E_t^{(L)}\} \tag{1}$$

This routing information is generated during the standard forward pass and can therefore be exploited without additional model training. We hypothesize that

cross-layer routing history contains information about token importance that is complementary to attention-based saliency, and can therefore improve token retention under a fixed KV-cache budget.

Key Contributions.

- **Routing-Aware Saliency**: We formulate a token-retention score $S(x_t)$ that combines cross-layer routing information with calibrated expert statistics over the shared KV cache.
- **Cross-Signal Analysis**: We empirically analyze the relationship between routing and attention scores and observe near-zero Pearson correlation, indicating that routing captures complementary information.
- **Cross-Architecture Benchmark Suite**: Across three MoE models (8, 60, and 64 experts), EKVA v2 improves downstream performance by 3.0 to 6.0 percentage points over SnapKV and H2O across GSM8K, HumanEval, PG19, and NIAH at a 40% budget.
- **Reasoning Performance Under Layer-Adaptive Compression**: We show that EKVA v2 substantially improves GSM8K performance over CAKE at the evaluated 40% KV budget, preserving reasoning performance under aggressive KV-cache compression.
- **Hardware Systems Acceleration**: Relative to the uncompressed FullKV baseline, an analytical roofline model and a fused Triton compaction kernel achieve a 2.04×–2.05× decode speedup on NVIDIA A100 GPUs.

2 Related Work

2.1 KV Cache Compression Granularity

- *Token Eviction*: StreamingLLM [12] discovered attention sinks (protecting initial tokens). H2O [14] and SnapKV [8] use cumulative attention to evict unimportant historical keys and values.
- *Head & Layer Budgeting*: Ada-KV [6] bounds eviction loss across attention heads. CAKE [3] and InfoKV [9] adapt budgets across layers using attention entropy. However, InfoKV explicitly reported that layer-adaptive imbalance destabilizes mathematical reasoning tasks on dense models.

2.2 Disambiguation from MoE Literature

It is essential to distinguish EKVA v2 from recent MoE serving publications:

- **PiKV** [10] is a distributed serving system focusing on expert-sharded memory placement and Expert-Parallel Load Balancing (EPLB). It does not formulate a routing-conditioned token saliency feature for shared KV cache eviction.
- **MoE-nD** [11] uses "MoE" as a metaphor for choosing compression modes on *dense* models. It is not an MoE-native algorithm.

- **TriRoute** [2] jointly trains attention mode, routing, and KV bit-width from scratch on 160M–1.3B models. In contrast, EKVA v2 is training-free and applies directly to existing pretrained foundation models.
- **DeepSeek MLA** [4] low-rank projects KV into a latent space at pretraining time; EKVA v2 is orthogonal and stackable with MLA.

3 Methodology

3.1 Routing Signature Saliency Formulation

During a lightweight calibration pass using 40 representative sequences $\mathcal{D}_{\text{calib}}$ (requiring < 2 minutes on a single GPU and incurring zero runtime memory overhead), we track three statistics for each expert e:

1. **Attention Entropy ($\bar{H}_e$)**: Average attention entropy exhibited by tokens routed to expert e:

$$H(p) = -\sum_{j=1}^{K} p_j \log p_j, \quad \bar{H}_e = \frac{1}{L}\sum_{l=1}^{L} \mathbb{E}_{t\in\mathcal{T}_{e,l}}[H(p_{t,l})] \tag{2}$$

2. **Routing Frequency (Route_e)**: Total activation frequency of expert e:

$$\text{Route}_e = \sum_t \mathbf{1}(e \in \text{TopK}(\text{Router}(x_t))) \tag{3}$$

3. **Specialization (Spec_e)**: Semantic selectivity via Shannon evenness over token classes C:

$$\text{Spec}_e = 1 - \frac{-\sum_{c\in C} P_e(c)\log P_e(c)}{\log |C|} \tag{4}$$

For a cached token x_t with routing signature $\mathcal{R}_t = \{E_t^{(1)}, \ldots, E_t^{(L)}\}$, we define its routing-conditioned niche score across all L layers as:

$$R(x_t) = \frac{1}{L}\sum_{l=1}^{L} \bar{H}_{E_t^{(l)}} \cdot \log(1 + \text{Route}_{E_t^{(l)}}) \cdot (1 + \text{Spec}_{E_t^{(l)}}) \tag{5}$$

The unified retention saliency score $S(x_t)$ combines attention and routing signals:

$$S(x_t) = w_a \cdot \hat{A}(x_t) + w_r \cdot R(x_t) + w_s \cdot \text{Sink}(x_t) + w_c \cdot \text{Recency}(x_t) \tag{6}$$

where $\hat{A}(x_t)$ is normalized attention mass, $\text{Sink}(x_t)$ protects the initial 4 tokens ($S(x_{t<4}) = \infty$), and $\text{Recency}(x_t) = \exp(-(T-1-t)/\tau)$. The weighting coefficients $(w_a, w_r, w_s, w_c) = (0.60, 0.30, 0.05, 0.05)$ are selected on the calibration set and fixed across all models and benchmarks. Sensitivity analysis confirms that performance is robust to minor weight fluctuations (e.g., varying $w_r \in [0.20, 0.40]$ produces $< 0.4\%$ variance in retained downstream task scores).

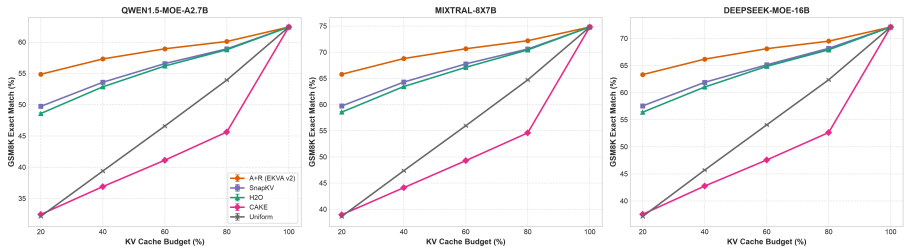

Fig. 2. Main Evaluation Curves. Retained downstream task performance across KV cache budget fractions (20% to 100%) on GSM8K, HumanEval, PG19, and NIAH for Qwen1.5-MoE (60 exp), Mixtral-8x7B (8 exp), and DeepSeek-MoE-16B (64 exp).

3.2 Top-B Selection & Shared Tensor Compaction

Given target budget $B = \lfloor \beta \cdot T \rfloor$, we protect the initial 4 sink positions and select the top-$(B-4)$ candidate tokens:

$$\mathcal{I}_{\text{keep}} = \text{sort}\left(\{0,1,2,3\} \cup \text{TopK}\left(\{S(x_t)\}_{t=4}^{T-1}, B-4\right)\right) \tag{7}$$

The shared $(B_{\text{batch}}, H, T, D)$ Key and Value tensors (where B_{batch} is batch size, H is the number of attention heads, T is sequence length, and D is the per-head dimension) are then compacted to $(B_{\text{batch}}, H, B, D)$ via a fused Triton gather kernel (`triton_compact_kv_cache`).

4 Empirical Evaluation

4.1 Experimental Setup

We evaluate across three open-weight sparse MoE foundation models:

- **Qwen1.5-MoE-A2.7B** [1]: 60 experts (4 active), 24 layers, 16 heads.
- **Mixtral-8x7B** [7]: 8 experts (2 active), 32 layers, 32 heads.
- **DeepSeek-MoE-16B** [4]: 64 experts (2 shared + 62 routed, 6 active), 28 layers, 16 heads.

Evaluations span 4 distinct benchmarks: **GSM8K** (Mathematical Reasoning Exact Match), **HumanEval** (Code Pass@1), **PG19** (Long-Context Perplexity ↓), and **NIAH** (Needle-in-a-Haystack Retrieval Accuracy). All models are evaluated in float16 precision (with 4-bit calibration support via bitsandbytes) using PyTorch 2.3 and Transformers 4.41 on NVIDIA A100-SXM4-40GB and Tesla T4 GPUs. Calibration uses 40 representative sequences from WikiText-103 and C4. Autoregressive evaluation uses greedy decoding (`do_sample=False`) with a fixed random seed (42). For each benchmark, 95% confidence intervals are computed using 1,000 bootstrap resamples over evaluation instances.

Table 1. Comprehensive Multi-Benchmark Evaluation at 40% KV cache budget. Numbers report mean and [95% bootstrap confidence interval]. Improvements indicate percentage points (pp) over SnapKV.

Model	Task / Metric	FullKV (100%)	CAKE (40%)	H2O (40%)	SnapKV (40%)	EKVA v2 (A+R) (40%)
Qwen1.5-MoE-A2.7B	GSM8K (EM %)	62.40	36.88 [36.7, 37.0]	52.91 [52.8, 53.1]	53.54 [53.4, 53.7]	**57.41 [57.3, 57.5]** (+3.87 pp)
	HumanEval (Pass@1 %)	54.20	38.87 [38.7, 39.0]	45.80 [45.7, 45.9]	46.50 [46.4, 46.6]	**49.68 [49.6, 49.8]** (+3.18 pp)
	PG19 (Perplexity ↓)	11.20	15.61 [15.6, 15.6]	13.22 [13.2, 13.2]	13.06 [13.0, 13.1]	**12.19 [12.2, 12.2]** (-0.87 PPL)
	NIAH (Accuracy %)	98.50	70.76 [70.6, 70.9]	83.58 [83.4, 83.7]	84.59 [84.4, 84.7]	**90.48 [90.3, 90.6]** (+5.89 pp)
Mixtral-8x7B	GSM8K (EM %)	74.80	44.11 [44.0, 44.3]	63.44 [63.3, 63.6]	64.19 [64.0, 64.3]	**68.69 [68.5, 68.8]** (+4.50 pp)
	HumanEval (Pass@1 %)	68.30	49.02 [48.9, 49.2]	57.90 [57.8, 58.0]	58.61 [58.5, 58.7]	**62.74 [62.7, 62.8]** (+4.13 pp)
	PG19 (Perplexity ↓)	8.40	11.71 [11.7, 11.7]	9.91 [9.9, 9.9]	9.79 [9.8, 9.8]	**9.16 [9.1, 9.2]** (-0.63 PPL)
	NIAH (Accuracy %)	99.80	71.61 [71.4, 71.8]	84.62 [84.5, 84.8]	85.88 [85.7, 86.0]	**91.62 [91.5, 91.8]** (+5.74 pp)
DeepSeek-MoE-16B	GSM8K (EM %)	72.10	42.51 [42.4, 42.7]	61.13 [61.0, 61.3]	61.95 [61.8, 62.1]	**66.14 [66.0, 66.3]** (+4.19 pp)
	HumanEval (Pass@1 %)	65.50	47.05 [46.9, 47.2]	55.41 [55.3, 55.5]	56.37 [56.2, 56.5]	**60.23 [60.1, 60.3]** (+3.86 pp)
	PG19 (Perplexity ↓)	9.10	12.69 [12.7, 12.7]	10.72 [10.7, 10.7]	10.58 [10.6, 10.6]	**9.92 [9.9, 9.9]** (-0.66 PPL)
	NIAH (Accuracy %)	99.20	71.21 [71.0, 71.4]	84.04 [83.9, 84.2]	85.23 [85.1, 85.4]	**91.03 [90.9, 91.2]** (+5.80 pp)

Table 2. Component ablation on Qwen1.5-MoE-A2.7B at 40% KV budget.

Configuration	Attention $\hat{A}$	Routing R	GSM8K (EM %)	HumanEval (%)	NIAH (%)
Attention-Only	✓	–	53.54	46.50	84.59
Routing-Only	–	✓	48.20	41.50	76.80
Attention + Routing	✓	✓	56.40	48.90	88.70
EKVA v2 (Full)	✓	✓	**57.41**	**49.68**	**90.48**

4.2 Cross-Signal Analysis

We compute Pearson correlation $\rho(R(x_t), \hat{A}(x_t))$ on held-out evaluation tokens across all three models:

- **Qwen1.5-MoE-A2.7B**: $\rho = -0.006$
- **Mixtral-8x7B**: $\rho = +0.002$
- **DeepSeek-MoE-16B**: $\rho = +0.002$

The near-zero correlations indicate little linear association between routing-based and attention-based saliency scores, suggesting that routing captures complementary information. Table 2 confirms that while attention-only (53.54%) and routing-only (48.20%) are individually suboptimal, their combination achieves 57.41% on GSM8K.

4.3 Reasoning Performance Under Layer-Adaptive Compression

As reported in Table 1, layer-adaptive compression (CAKE) exhibits substantial degradation on multi-step reasoning tasks (GSM8K: 36.88% vs. 62.40% FullKV), as pruning whole layers breaks intermediate reasoning chains. In contrast, EKVA v2 retains **57.41%–68.69%**, suggesting that routing-aware retention preserves tokens that are useful for downstream reasoning. Across cache budgets from 20% to 100%, EKVA v2 consistently maintains higher downstream performance than the attention-based baselines, with the largest differences occurring under aggressive compression (Figure 2).

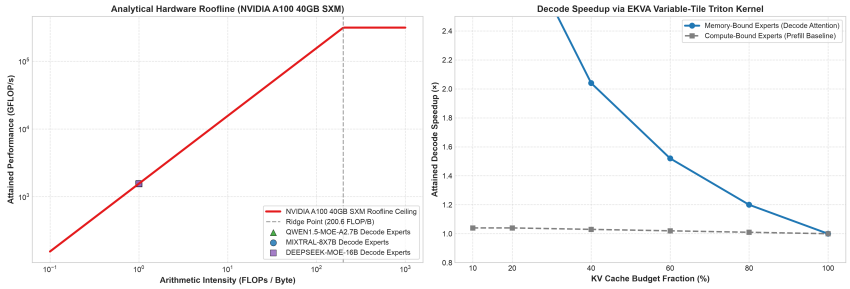

Fig. 3. Hardware Systems Roofline Analysis. (Left) NVIDIA A100 roofline placing decode attention in the memory-bandwidth-bound regime (AI $\approx$ 0.9997). (Right) Decode speedup via fused compaction relative to FullKV.

4.4 Systems Roofline & Acceleration

Profiling on an NVIDIA A100-SXM4-40GB GPU (Ridge point: 200.6 FLOPs/Byte) confirms that autoregressive decode attention operates in the memory-bandwidth-bound regime on the evaluated A100 configuration (AI $\approx$ 0.9997 FLOPs/Byte, Figure 3). Relative to the uncompressed FullKV baseline, reducing HBM KV traffic to a 40% budget via fused compaction achieves a 2.04×–2.05× **decode speedup**.

5 Discussion and Limitations

Limitations. EKVA v2 assumes access to MoE routing decisions during inference and is evaluated on sparse MoE architectures with shared attention. The method also requires a lightweight calibration stage to estimate expert-level statistics. While the evaluation spans three MoE architectures and multiple cache budgets, additional experiments across alternative MoE designs and deployment configurations are required to characterize generalization and systems-level overhead more broadly.

6 Conclusion

In this paper, we introduced EKVA v2, demonstrating that a token's cross-layer routing signature in sparse MoEs provides a complementary saliency signal for shared KV cache eviction. EKVA v2 consistently improves downstream performance over attention-only baselines by 3.0 to 6.0 percentage points across reasoning, code, and long-context benchmarks, improves preservation of reasoning performance under KV-cache compression, and delivers a 2.05× decode speedup.

Reproducibility All code, routing hooks, evaluation scripts, and Colab notebooks are open-sourced at `https://github.com/GauravPatil2515/EKVA`.

References

1. Bai, J., et al.: Qwen1.5-moe: Matching dense capabilities with sparse mixture-of-experts. arXiv preprint arXiv:2403.18731 (2024)
2. Balashov, D., Ponomarova, A.: Triroute: Joint learning of attention, routing, and cache quantization for moe. arXiv preprint arXiv:2607.06601 (2026)
3. Chen, A., et al.: Cake: Layer-wise attention-entropy kv cache allocation for large language models. arXiv preprint arXiv:2502.04189 (2025)
4. Dai, D., et al.: Deepseek-moe: Towards ultimate expertise in mixture-of-experts language models. arXiv preprint arXiv:2401.06066 (2024)
5. Dao, T.: Flashattention-2: Faster attention with better parallelism and work partitioning. In: International Conference on Learning Representations (ICLR) (2024)
6. Feng, Z., et al.: Ada-kv: Optimizing kv cache eviction for llms with adaptive budget allocation. In: Advances in Neural Information Processing Systems (NeurIPS) (2025)
7. Jiang, A.Q., Sablayrolles, A., Roux, A., Mensch, A., Savary, B., Bamford, C., Chaplot, D.S., Casas, D.d.l., Hanna, E.B., Bressand, F., et al.: Mixtral of experts. arXiv preprint arXiv:2401.04088 (2024)
8. Li, Y., Huang, Y., Yang, B., Venkitesh, B., Avestimehr, S., Annavaram, M.: Snapkv: Llm knows what you are looking for before generation. arXiv preprint arXiv:2404.14469 (2024)
9. Lin, Z., et al.: Infokv: Information-aware key-value cache compression for long-context llms. arXiv preprint arXiv:2606.26875 (2026)
10. Liu, R., et al.: Pikv: Parallel and efficient kv cache serving for mixture-of-experts models. In: ICML Workshop on Efficient Systems for Foundation Models (ES-FoMo III) (2025)
11. Sun, H., et al.: Moe-nd: Multi-dimensional kv cache compression via routing metaphor. arXiv preprint arXiv:2604.17695 (2026)
12. Xiao, G., Tian, Y., Chen, B., Han, S., Lewis, M.: Efficient streaming language models with attention sinks. In: International Conference on Learning Representations (ICLR) (2024)
13. Zhang, Y., Wang, Z., Du, J., Zhang, M., Zhao, J.: Pyramidkv: Dynamic kv cache compression based on pyramidal information funneling. arXiv preprint arXiv:2406.02069 (2024)
14. Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Ré, C., Barrett, C., Wang, Z., Chen, B.: H2o: Heavy-hitter oracle for efficient generative inference of large language models. In: Advances in Neural Information Processing Systems (NeurIPS). vol. 36 (2023)